

Software Engineering

Operations and Maintenance

Post Mortem Analysis

Murphy's Law

What can go wrong, will go wrong.

DevOps Principle

“You build it, you run it.”

Post Mortem Analysis

Important: blameless feedback culture, no finger pointing

Goal: eliminate the problem in the system that caused the failure

Who: everyone involved, e.g. Infrastructure Engineers, Software Engineers, Quality Engineers...

When: after incident is resolved but shortly after so the memory is still fresh

How: systematically with a template, synchronous meeting

Outcome: written PMA with action steps

Guiding question: how can we not make the same mistake twice?

Post Mortem Analysis

Title (incident #)

- Date
- Authors
- Summary
- Impact
- Root Causes
- Trigger
- Resolution
- Detection
- Action Items
- Lessons Learned
 - What went well
 - What went wrong
 - Where we got lucky
- Timeline

Exercise - 15 mins

In groups of 3-5:

Use this [example template](#) on the Launch Scenario

Put your PMA in the Teams Chat

Exercise: PMA on Launch Scenario

MentorHive's Product Team planned a new "Career Goals" feature meant to help mentees quickly set structured career plans. They had informed Marketing a few weeks ago that the feature would be ready by Wednesday.

The Friday before the planned launch date, QA had flagged several workflow inconsistencies. Engineering was working on fixing it from Monday on and believed the launch date was postponed without aligning with the Product Team, believing the QA Team would take care of it.

Tuesday night Marketing pushed out the scheduled announcement across email and LinkedIn, excited to finally promote the feature. On Wednesday morning, thousands of users clicked on the provided link through to try the new "Career Goals" — only to find the feature failing halfway through the workflow.

Support was flooded with complaints. Engineering woke up to incident alerts showing mass errors from users attempting to create "Career Goals". Confused, they asked Product why users were accessing a feature still behind a partially active feature flag. Product assumed Engineering had already fixed the workflow issues found by QA and enabled the feature because they hadn't asked to postpone the launch. Marketing was never informed that there were issues that might delay the launch.

By midday, the Engineering Team turned off access fully and Marketing published an apology. In less than six hours:

- Over 12,000 users encountered broken workflows
- Support tickets spiked 8x
- Social media mocked MentorHive

Monitoring and Observability

Application Performance Monitoring – Motivation

- While applications are running we want to be able to examine their status
 - Are they running into failures?
 - Do we have performance issues?
 - Do we have networking bottlenecks?
- This is especially interesting when we have hundreds of services and lots of applications
- The purpose of Monitoring is to
 - Watch and record system and service behaviour
 - Trace individual calls throughout the systems landscape
 - Provide useful reports for staff
 - Alert on misbehaviour
 - **Gather information to improve the software continuously**

Alerts with Thresholds

Alerts with thresholds are key components of monitoring and observability practices in software engineering. They help ensure that systems are running as expected and provide mechanisms to respond quickly when something goes wrong.

Alerts are notifications triggered when certain conditions or events occur in a system. They serve as a proactive mechanism to inform operators, developers, or automated systems about potential issues that need attention. Alerts can be configured to notify through various channels such as email, SMS, Slack, or integrated with incident management systems.

Trigger Conditions: Alerts are based on specific conditions or rules that monitor metrics, logs, or traces. For example, an alert might trigger if the CPU usage exceeds 90% for more than 5 minutes.

Severity Levels: Alerts can have different levels of severity, such as critical, warning, or informational, depending on the impact of the issue.

Actionable: Good alerts should be actionable, meaning that they provide enough information to understand the problem and initiate a response.

Alerts with Thresholds

Thresholds define specific values or conditions that, when crossed, trigger alerts. They act as the boundaries between normal and abnormal behavior for a particular metric or log entry.

Static Thresholds: These are predefined, fixed values. For example, a static threshold might be set so that an alert triggers when memory usage exceeds 80%.

Dynamic Thresholds: These adjust based on trends or historical data. For example, if a system normally operates with 60-70% CPU usage, a dynamic threshold might trigger an alert only if the usage exceeds the normal range significantly, accounting for expected variations.

Anomalies: Thresholds can be used to detect anomalies, which are deviations from expected patterns. For example, a sudden drop in user activity might trigger an alert if it falls below a certain threshold for a specific time frame.

Observability goes beyond Monitoring

Goals of Observability

System Reliability: Observability helps in identifying and diagnosing issues before they become critical, thereby improving system reliability and availability.

Performance Optimization: By understanding where bottlenecks or inefficiencies are, teams can optimize the performance of their systems.

Debugging: When issues occur, observability tools help engineers quickly identify the root cause by providing detailed insights into the system's behavior.

Predictive Maintenance: Observability data can be used to anticipate and prevent potential problems, such as resource exhaustion or unexpected traffic spikes.

Observability vs. Monitoring

While observability and monitoring are closely related, they are not the same:

Monitoring involves tracking **predefined** metrics and logs to detect known issues. It's about collecting data and alerting on specific thresholds.

Observability is broader, focusing on understanding the system's behavior through analysis of its outputs. It allows engineers to **ask new questions and explore the system's state** in an open-ended way, even when the problem was not anticipated.

Legacy Code, Refactoring and Tech Debt

Legacy Code

Code that someone didn't write originally
but inherited the ownership for

Refactoring and Rewrites

Refactoring is the process of restructuring existing code without changing its external behavior.

The primary goal of refactoring is to improve the code's internal structure, making it cleaner, more efficient, and easier to understand and maintain.

Rewriting code means to start on a green field, often also making all architectural choices newly: what tech stack to use, what features to implement, how to test it...

The fallacy here is often to assume that starting from scratch will create a better solution sooner than the old one was created.

Personal opinion: prefer Refactoring over Rewrites

Refactoring Strategies

Hot Spot Refactoring

Refactor the code that

- Is most often changed
- Has the highest complexity
- You find the most bugs in (bug clusters)

Campground Rule Refactoring

- Align on a Tech Vision for larger refactorings
- Create a PoC (proof of concept) in one part of the code
- Refactor the rest of the code incrementally whenever you change it

Preparatory Refactoring

- When creating the technical concept for a new user story, check the architecture
- If it is not fit for the change, refactor it so that the new feature is easier to implement

Technical Debt

Shortcuts that were taken to ship features faster

- Architecture does not fit the requirements
- Quality Criteria not met (anymore)
 - Specifically Maintainability, e.g. Automated Testing
- Software eroded over time
 - Updates of systems and dependencies (e.g. libraries) not made

You pay interest fees on Debt, this also applies to Tech Debt

- Implementing new features takes more time
- In Software with Tech Debt there will more likely new bugs

Tech Debt needs to be paid back by Refactoring and Continuous Improvements

- Otherwise it slows down the business

The opposite of Tech Debt is Technical Excellence

What is the economic impact of software architecture (aka design)?

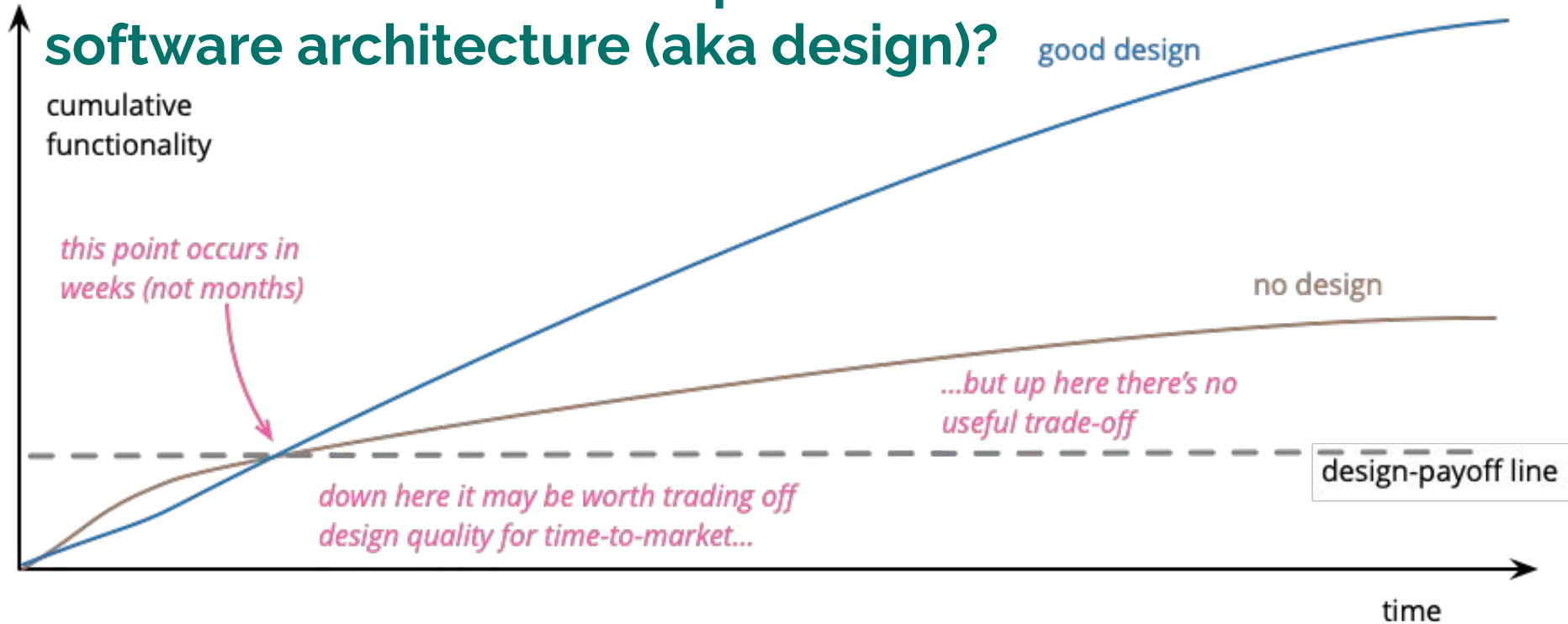


Image source: <https://martinfowler.com/bliki/DesignStaminaHypothesis.html>

